

Usando *type hint*

Type hints são anotações que dizem qual tipo cada variável, parâmetro e retorno deveria ter: `def media(notas: list[float]) -> float`. O Python não as confere ao rodar, mas editores e verificadores como o `mypy` usam essas anotações para achar erros antes da execução e para completar código. Nesta aula: a sintaxe básica, coleções, valores opcionais, funções como parâmetro, apelidos e genéricos, dataclasses e o `mypy` em ação.

nome: tipo · -> retorno

`list[int]` · `dict[str, float]`

`str | None`

`Callable`

type · genéricos `[T]`

`dataclass`

`mypy`

Formação Claude Code 2026 · Baseado em docs.python.org (typing; Type alias statements; Generic functions; dataclasses; Annotations Best Practices), PEP 484, PEP 604, PEP 695 e PEP 649, e na documentação do mypy, consultados em outubro de 2026 · Exemplos testados com Python 3.14 e mypy 2.4 em `exemplos/` · Saídas reais

Anotando parâmetros, *retorno e variáveis*

basico.py

```
def preco_final(valor: float, desconto: float = 0.0) -> float:  
    return round(valor * (1 - desconto), 2)
```

```
cliente: str = "Ana"  
itens: int = 3
```

```
print(preco_final(200, 0.15))  
print(preco_final.__annotations__)
```

SAÍDA

```
170.0  
{'valor': <class 'float'>, 'desconto': <class 'float'>, 'return':  
<class 'float'>}
```

- ✓ `parâmetro: tipo` e `-> tipo` depois dos parênteses para o retorno
- ✓ Variáveis também: `cliente: str = "Ana"`
- ✓ As anotações ficam guardadas na função (`__annotations__`) e servem de documentação
- ✓ No Python 3.14, elas são avaliadas só quando alguém as consulta, sem custo na importação

O Python *não confere* ao rodar

nao_valida.py

```
def dobrar(n: int) -> int:
    return n * 2

print(dobrar(21))
print(dobrar("ab"))      # o Python não confere os tipos
print(dobrar([0]))
```

SAÍDA

```
42
abab
[0, 0]
```

- ✓ Anotar `n: int` não impede passar uma string ou uma lista
- ✓ Quem confere é uma ferramenta separada (`mypy` , `pyright`) ou o editor, antes de rodar
- ✓ Para validar dados de fora em tempo de execução, continue validando (aula 07)

Type hint é um contrato para quem lê e para as ferramentas, não uma trava do interpretador.

list, dict, tuple *e set* com o tipo dos itens

colecões.py

```
def media(notas: list[float]) -> float:
    return sum(notas) / len(notas)

def por_aluno(
    registros: list[tuple[str, float]],
) -> dict[str, float]:
    return {nome: nota for nome, nota in registros}

tags: set[str] = {"python", "tipos"}
print(media([7.5, 9.0, 8.0]))
print(por_aluno([("ana", 9.5), ("bia", 8.0)]))
print(sorted(tags))
```

SAÍDA

```
8.166666666666666
{'ana': 9.5, 'bia': 8.0}
['python', 'tipos']
```

ANOTAÇÃO

SIGNIFICA

`list[float]`

lista de números decimais

`dict[str, float]`

chave texto, valor decimal

`tuple[str, float]`

par: texto e decimal, nessa ordem

`tuple[int, ...]`

tupla de inteiros, qualquer tamanho

`set[str]`

conjunto de textos

Desde o Python 3.9, use os tipos nativos (`list[int]`) em vez de `typing.List` .

Pode vir *None*

opcional.py

```
def buscar_email(nome: str) -> str | None:    # pode faltar
    cadastro = {"ana": "ana@site.com"}
    return cadastro.get(nome)

email = buscar_email("bia")
if email is None:
    print("sem e-mail cadastrado")
else:
    print(email.upper())    # aqui o verificador sabe que é str
```

SAÍDA

```
sem e-mail cadastrado
```

- ✓ `str | None` : devolve um texto ou `None` (equivale a `Optional[str]`)
- ✓ O verificador obriga a tratar o `None` antes de usar `.upper()`
- ✓ Depois do `if email is None` , ele sabe que no `else` é `str`

Esse é um dos maiores ganhos dos type hints: pegar o famoso "NoneType has no attribute" antes de rodar.

Funções como parâmetro: *Callable*

funcoes.py

```
from collections.abc import Callable, Iterable

def aplicar(
    funcao: Callable[[int], int],
    valores: Iterable[int],
) -> list[int]:
    return [funcao(v) for v in valores]

print(aplicar(lambda n: n * n, range(4)))
```

SAÍDA

```
[0, 1, 4, 9]
```

- ✓ `Callable[[int], int]` : função que recebe um `int` e devolve `int`
- ✓ `Iterable[int]` aceita lista, tupla, `range`, gerador... (aulas 01 a 04)
- ✓ Anote os parâmetros com o tipo mais **geral** que funciona; o retorno, com o mais específico

type Ponto e funções [T]

alias_generico.py

```
type Ponto = tuple[float, float]      # apelido (Python 3.12+)

def distancia(a: Ponto, b: Ponto) -> float:
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

def primeiro[T](itens: list[T]) -> T:    # T: tipo genérico
    return itens[0]

print(distancia((0, 0), (3, 4)))
print(primeiro(["x", "y"]), primeiro([10, 20]))
```

SAÍDA

```
5.0
x 10
```

- ✓ `type Ponto = ...` dá um nome a um tipo longo (Python 3.12+)
- ✓ `def primeiro[T]:` `T` é um tipo variável. Com lista de `str`, devolve `str`; com `int`, `int`
- ✓ O verificador acompanha o tipo do argumento até o retorno

Classes de dados *a partir das anotações*

dataclass_tipos.py

```
from dataclasses import dataclass, field

@dataclass
class Pedido:
    cliente: str
    itens: list[str] = field(default_factory=list)
    pago: bool = False

p = Pedido("Ana", ["livro"])
print(p)
print(Pedido("Bia"))
```

SAÍDA

```
Pedido(cliente='Ana', itens=['livro'], pago=False)
Pedido(cliente='Bia', itens=[], pago=False)
```

- ✓ `@dataclass` lê as anotações e cria `__init__`, `__repr__` e `__eq__`
- ✓ Campos com valor padrão viram parâmetros opcionais
- ✓ `field(default_factory=list)` cria uma lista nova para cada objeto (nunca use `= []`)

Aqui as anotações mudam o comportamento: a dataclass depende delas para saber quais campos existem.

O verificador *acha os erros*

com_erros.py

```
def media(notas: list[float]) -> float:
    return sum(notas) / len(notas)

def saudacao(nome: str) -> str:
    return "Olá, " + nome

media(["8", "9"])           # strings em vez de números
saudacao(42)                # int em vez de str
total: int = media([7.0, 8.0]) # float guardado como int
```

- ✓ Instale com `pip install mypy` e rode `mypy arquivo.py`
- ✓ Nada roda: o mypy só lê o código e compara com as anotações
- ✓ Editores como VS Code e Cursor mostram os mesmos avisos enquanto você digita

SAÍDA DO MYPY

```
$ mypy com_erros.py
com_erros.py:9: error: List item 0 has incompatible type "str";
expected "float" [list-item]
com_erros.py:9: error: List item 1 has incompatible type "str";
expected "float" [list-item]
com_erros.py:10: error: Argument 1 to "saudacao" has incompatible type
"int"; expected "str" [arg-type]
com_erros.py:11: error: Incompatible types in assignment (expression
has type "float", variable has type "int") [assignment]
Found 4 errors in 1 file (checked 1 source file)

$ mypy ex_estoque.py
Success: no issues found in 1 source file
```

Cada erro traz a linha e um código entre colchetes (`[arg-type]`), útil para pesquisar.

Estoque *tipado*

Enunciado: anote as funções `repor` e `em_falta` de um estoque `dict[str, int]`, com um apelido de tipo, e confira com o mypy.

Uma resposta possível

```
type Estoque = dict[str, int]

def repor(estoque: Estoque, item: str, qtd: int = 1) -> Estoque:
    return {**estoque, item: estoque.get(item, 0) + qtd}

def em_falta(estoque: Estoque) -> list[str]:
    return [item for item, qtd in estoque.items() if qtd == 0]

atual: Estoque = {"caneta": 0, "papel": 12}
novo = repor(atual, "caneta", 5)
print(novo, em_falta(atual), em_falta(novo))
```

SAÍDA

```
{'caneta': 5, 'papel': 12} ['caneta'] []
```

- ✓ O mypy aprovou o arquivo: "Success: no issues found"
- ✓ `repor` devolve um estoque novo, sem alterar o antigo (aula 13)
- ✓ Desafio: chame `repor(atual, 5, "caneta")` e veja o que o mypy diz

Teste o que *ficou*

1. `def f(n: int) → int`, chamada com `f("a")`, ao rodar:

A) Dá `TypeError` por causa da anotação

B) Roda; a anotação não é conferida

C) Converte "a" para int

D) Não compila

B. Quem avisa é o mypy ou o editor.

2. Função que pode devolver texto ou nada:

A) `→ str`

B) `→ None`

C) `→ str | None`

D) `→ any`

C.

3. Dicionário de nomes para notas:

A) `dict[float, str]`

B) `dict[str, float]`

C) `list[str, float]`

D) `dict(str, float)`

B.

4. Para que serve o mypy?

A) Executar mais rápido

B) Verificar os tipos sem rodar o código

C) Formatar o código

D) Instalar pacotes

B.

Tipos que documentam *e previnem erros*

Anote parâmetros, retornos e variáveis com `: tipo` e `-> tipo`, use `list[int]`, `dict[str, float]`, `str | None` e `Callable`, e dê nomes com `type`. O Python não confere nada ao rodar: o `mypy` e o editor fazem isso antes, e as dataclasses aproveitam as anotações para montar a classe.

`: tipo · -> tipo`

`list[int]`

`X | None`

`Callable`

`type · [T]`

`mypy`

Próxima aula → 15 · Usando Zip e Enumerate

Fontes: docs.python.org/3/library/typing.html; docs.python.org/3/reference/simple_stmts.html#the-type-statement; docs.python.org/3/reference/compound_stmts.html#generic-functions; docs.python.org/3/library/dataclasses.html; docs.python.org/3/howto/annotations.html; peps.python.org/pep-0484, [pep-0604](https://peps.python.org/pep-0604), [pep-0695](https://peps.python.org/pep-0695) e [pep-0649](https://peps.python.org/pep-0649); mypy.readthedocs.io, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4; verificação com mypy 2.4.0.